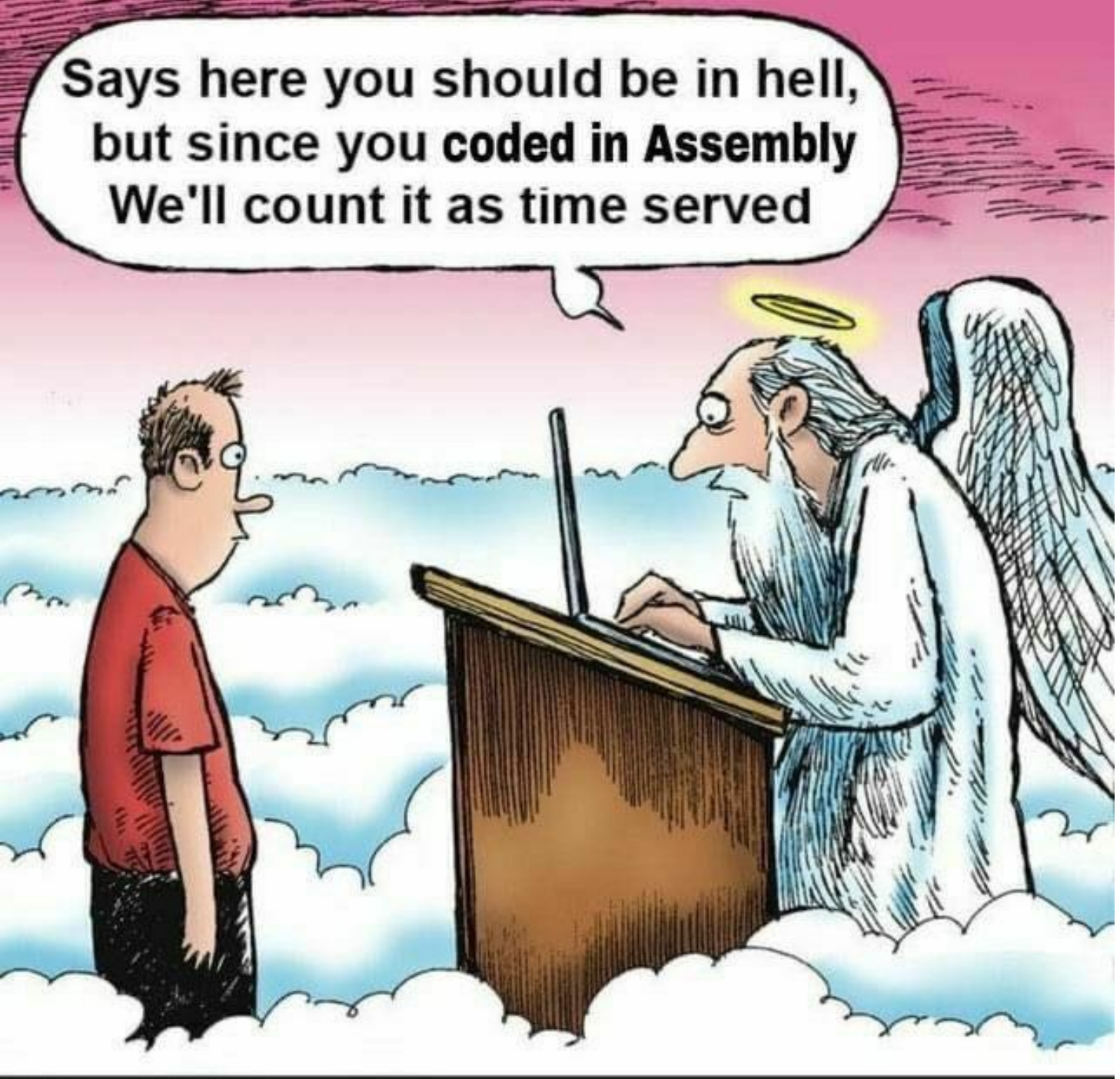
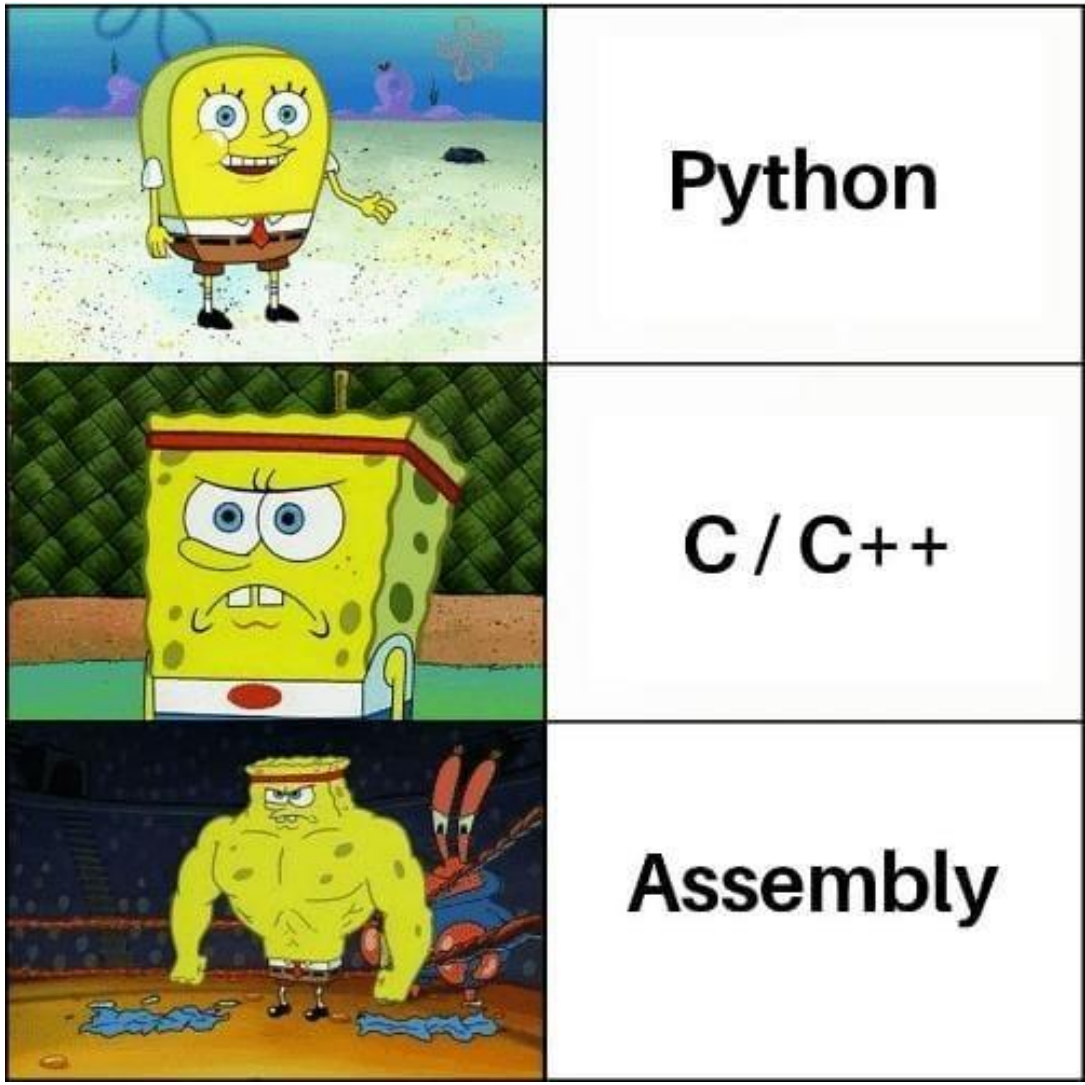
2. Část - Exploitace

Agenda

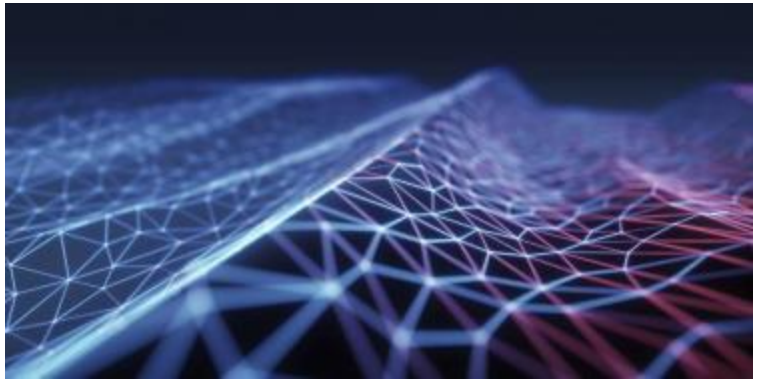
- Úvod - Potřebný SW, CTF platforma
- Buffer Overflow
- x32 vs. x64 arch
- ROP Syscalls
- Ret2libc w active ASLR
- Stack canaries
- SEH
- Egghunter




Says here you should be in hell,
but since you coded in Assembly
We'll count it as time served
ProgrammerHumor.io


Python
C / C++
Assembly

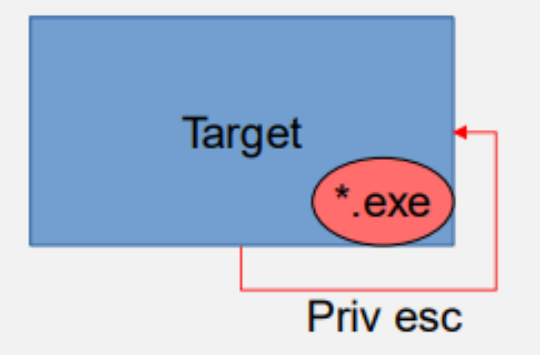
Co je exploit?

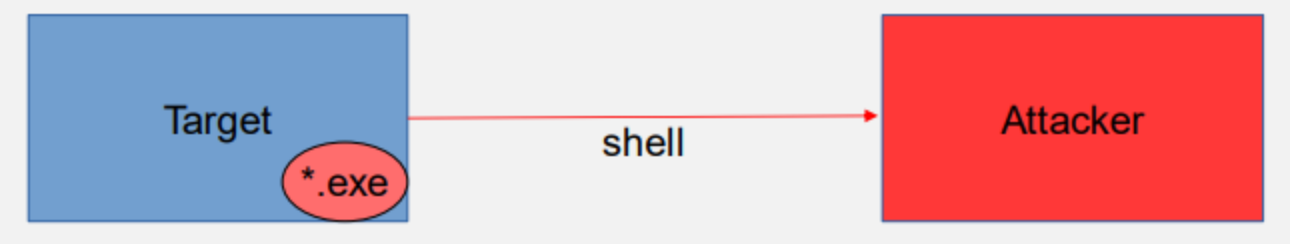


- Z anglického slovesa "to exploit"
  - využít něco pro svůj prospěch
- SW / spleť dat / sekvence příkazů, která znežije bug nebo vuln
- Používáme je k přístupu do sítě/zařízení, k PrivEsc, případně k DDoS

Exploit - Typy útoku

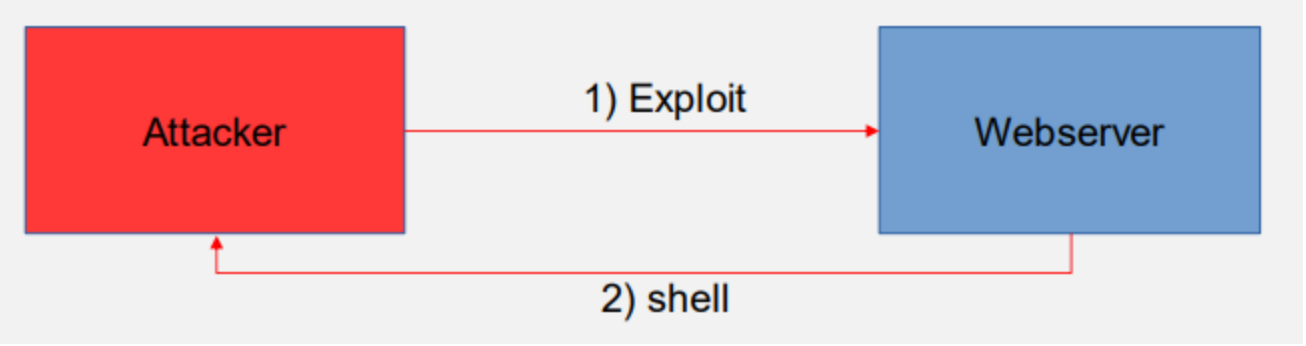
- Local Exploit
  - Běžící local exec file v systému
  - Možný následek:
    - PrivEsc (user -> admin)
    - RevShell (vzdálený přístup pro útočníka)



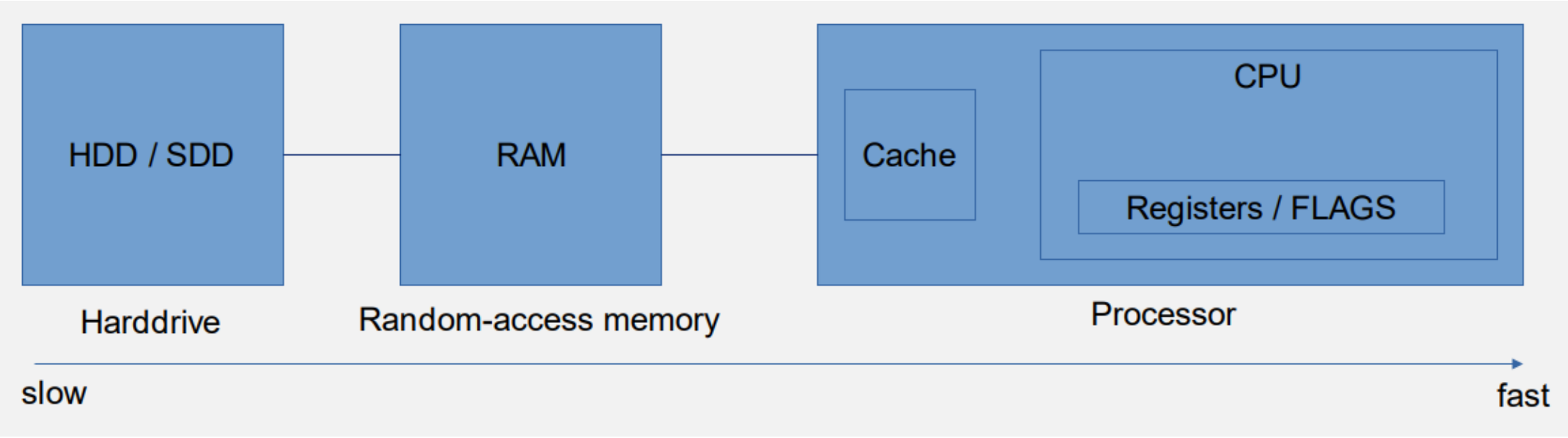


Exploit - Typy útoku

- Remote Exploit
  - Zaslání paketů webserveru k exploitaci vuln
  - Možný následek:
    - Server spustí příkaz, který by neměl



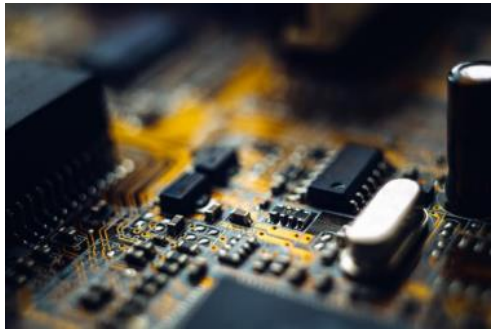
Fyzická paměť

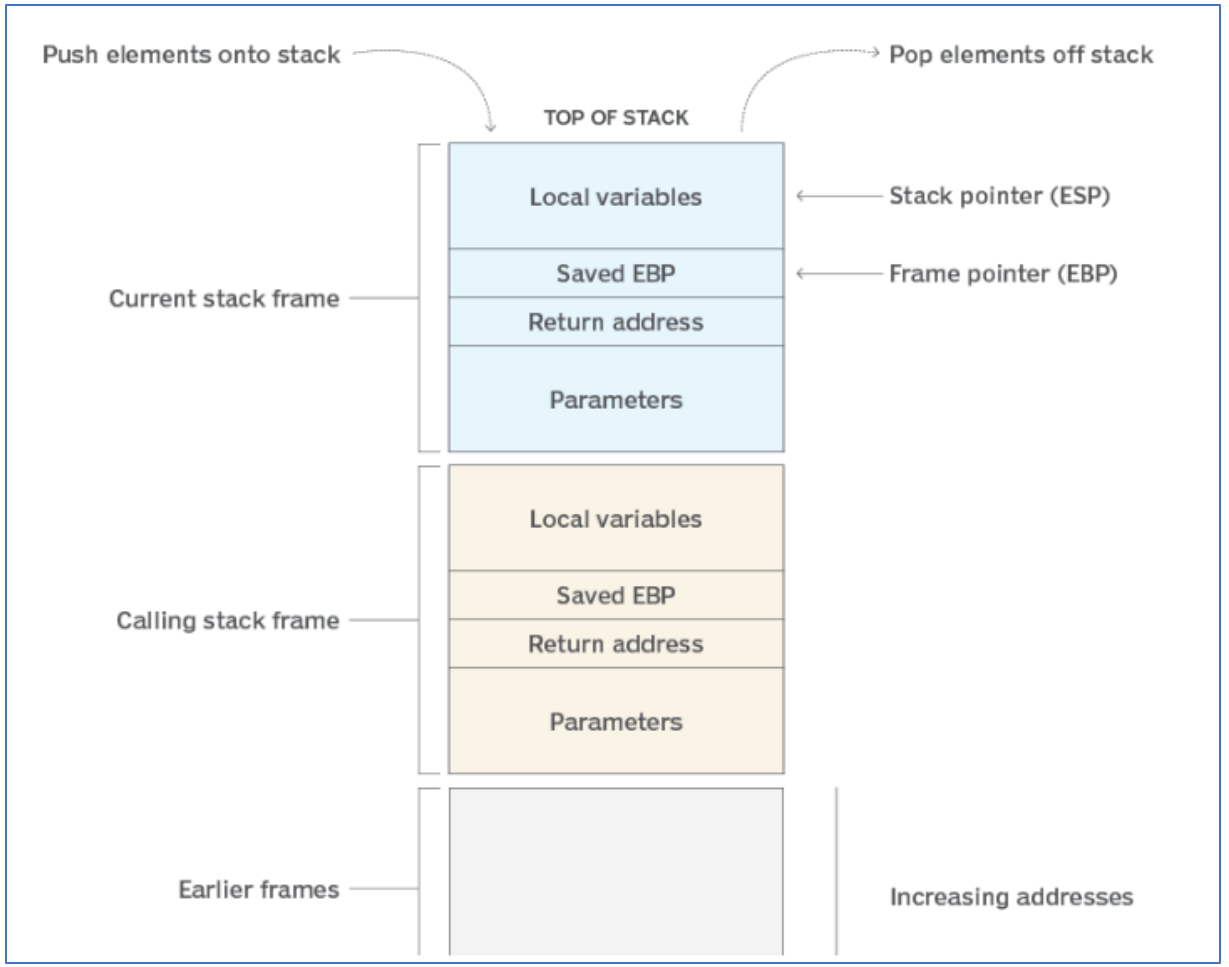


x86-CPU: Registry

- Registry = úložiště pro CPU
  - Výhoda -> rychlost toku dat
  - Nevýhoda -> malá kapacita

- Některé mají speciální funkce





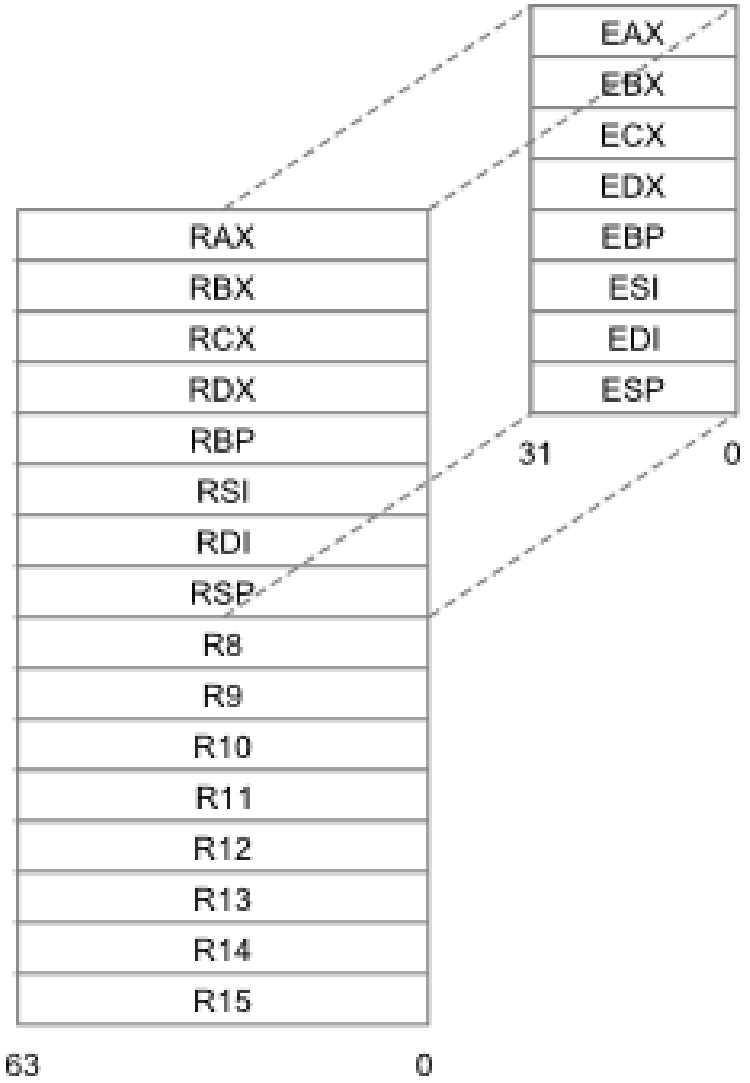
x86-CPU: Registry

- AX: Accumulator -> akumulátor pro operandy a výsledná data
- BX: Base Pointer -> ukazuje na data v segmentu DS
- CX: Counter Pointer -> čítač pro řetězcové a smyčkové operace
- DX: Data Pointer -> ukazatel dat (I/O ukazatel)
- SI: Source Pointer -> zdrojový ukazatel pro operace se stringy
- DI: Destination Pointer -> cílový ukazatel pro operace se stringy
- SP: Stack Pointer -> ukazuje na začátek zásobníku
- BP: Stack Base Pointer -> ukazatel pro rámec zásobníku (stack frame)
- IP: Instruction Pointer -> ukazuje na další instrukci, která má být provedena

x86-CPU: Registry (32 / 64 bit)

- 32-bit
  - První písmeno "E" -> "Extended"
    - EAX, EBX, ECX…ESP, EIP
  - Velikost 32 bitů

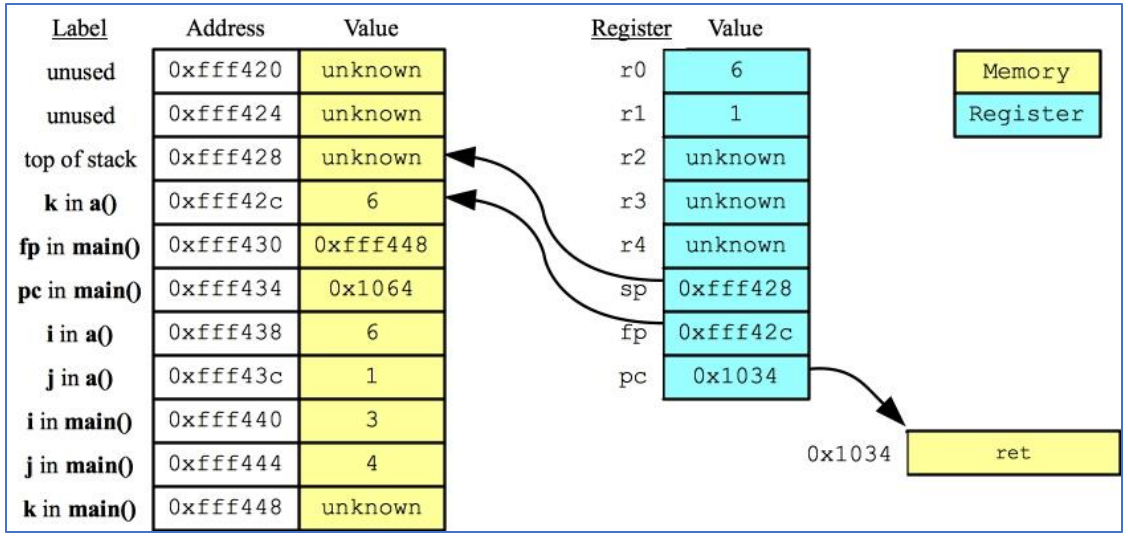
- 64-bit
  - První písmeno "R" -> "Register"
    - RAX, RBX, RCX…RSP, RIP
    - R8 – R15 (speciální registry)
  - Velikost 64 bitů



x86-CPU: Registry (32 / 64 bit)

- Registry mohou uchovávat v podstatě cokoliv

- Registr obsahuje nějakou HODNOTU
  - Ta je interpretována jako POINTER
    - ...a ten obsahuje ADRESU



Status Flags

- Uchovává výsledek nějaké operace (aritmických instrukcí)
- Dva nejdůležitější Status Flags:
  - CARRY FLAG -> nastaven, pokud instrukce přenáší hodnotu
  - ZERO FLAG -> nastaven, pokud je hodnota 0

- K čemu jsou?
  - Nastavení podmínek
    - např. podmínka: "jump if zero"
      - Pokud je hodnota instrukce 0, můžeš skočit na danou adresu

Endianness



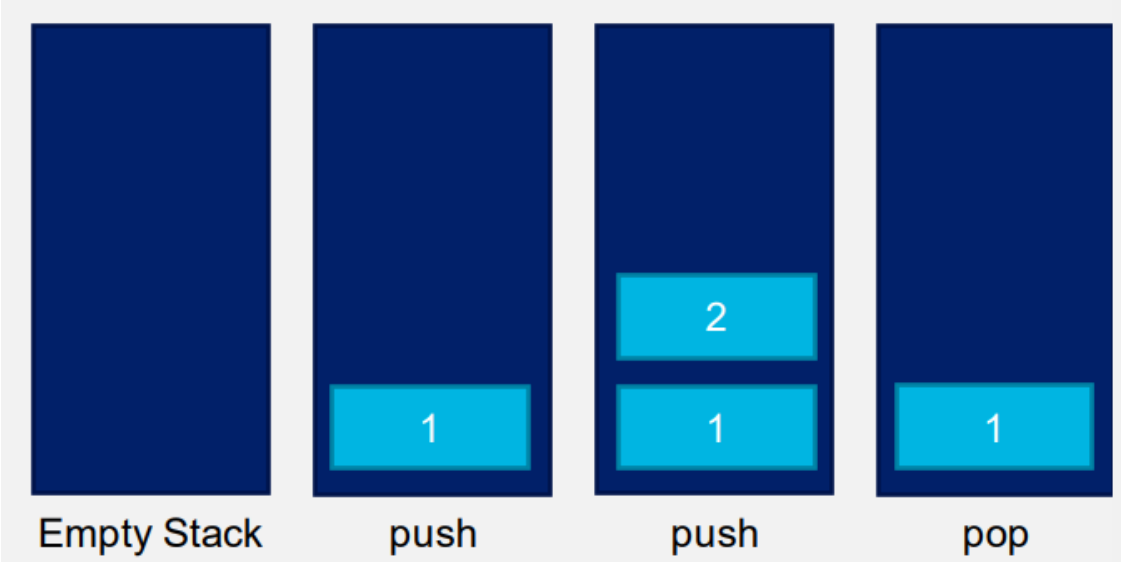
- 2 druhy zápisu adresy

- BIG-endian -> Most significant byte … Least significant byte
- LITTLE-endian -> Least significant byte … Most significant byte

Big Endian	01	02	03	04
Little Endian	04	03	02	01

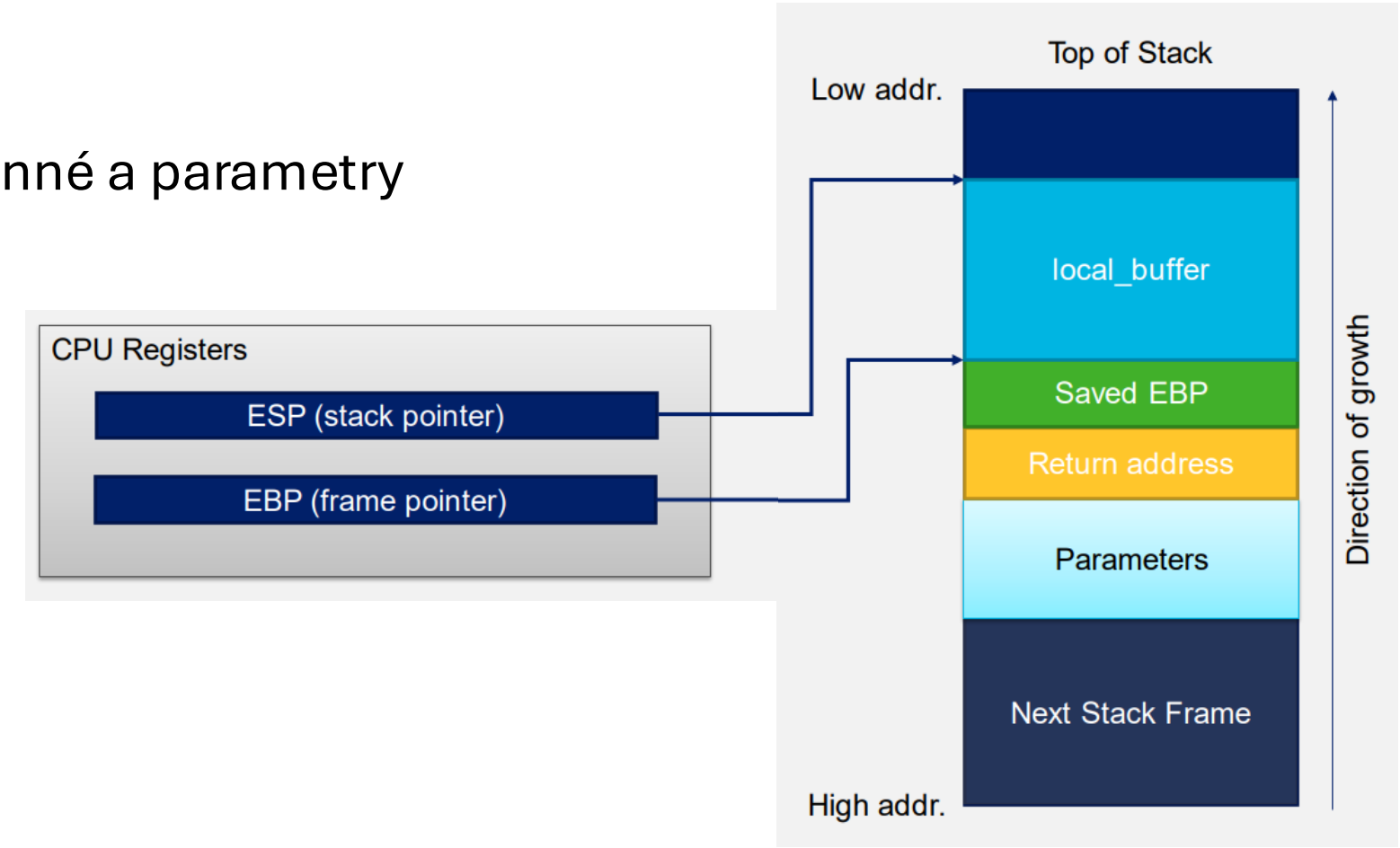
Stack

- Jednoduchá datová struktura
  - Úkol: uchovávat lokální proměnné
    - Po jejich použití je automaticky vymaže
- Rychlé, ale jen pro malé množství dat
- LIFO pro řazení do fronty
- Nové instrukce se řadí nahoru
- Stavební blok -> Stack Frame

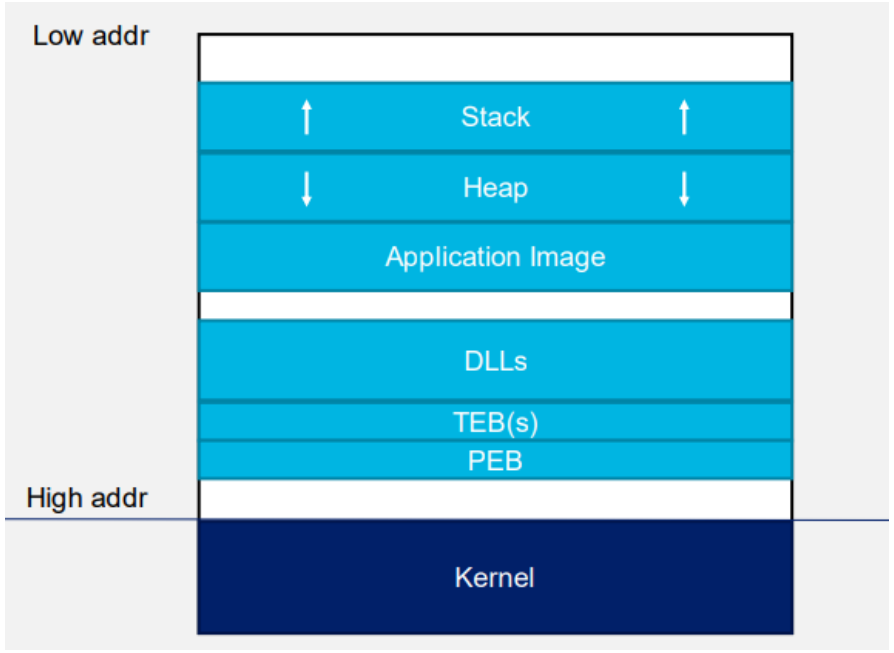


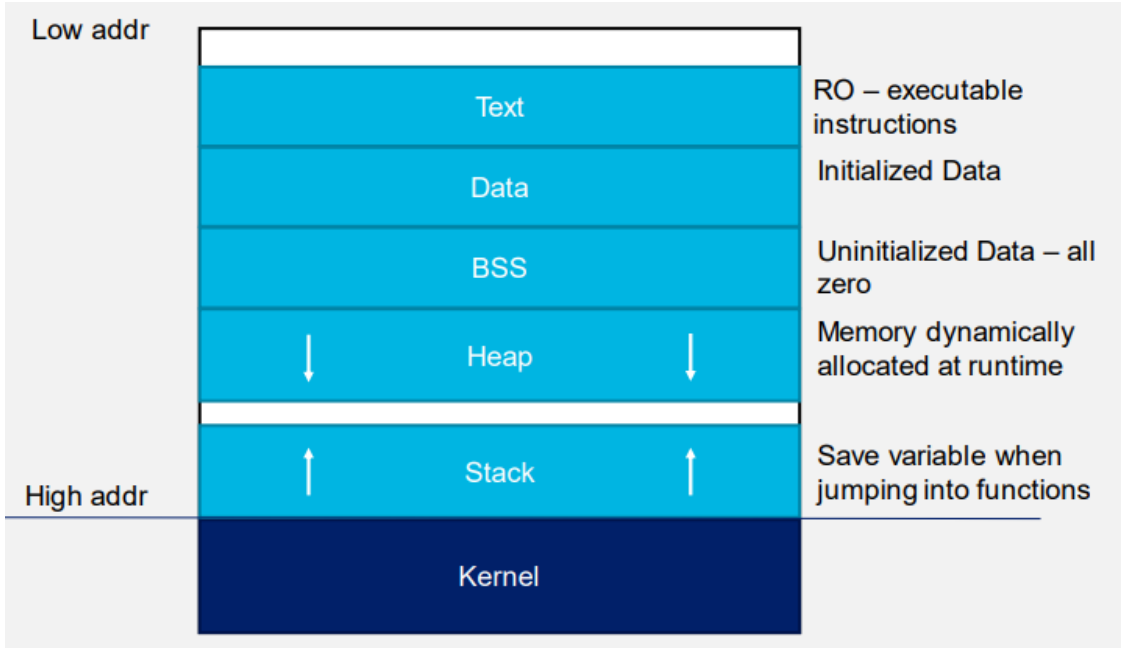
Stack Frame

- Obsahuje lokální proměnné a parametry
- Ukládá return adresy

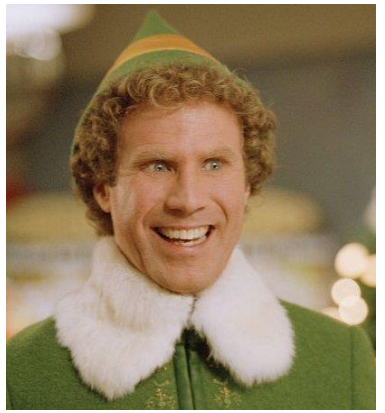


Memory layout - Windows X Linux





ELF soubory



- = Executable and Linkable Format
- Standardní formát binárních souborů pro Unix na procesorech x86
- Obsahuje 4 hlavní části s různým přístupem pro čtení, zápis a spouštění:
  - .text: spustitelný kód
  - .data: globální tabulky, proměnné atd.
  - .rodata: pouze pro čtení, řetězce
  - .bss: neinicializovaná pole a proměnné, vyplněné nulami

Compiler

Hello.c

```c
#include <stdio.h>

int main()
{
    printf("Hello MEXE");
    return 0;
}
```

HLL

Hello.s

```
    .file   "hello.c"
    .text
    .section    .rodata
.LC0:
    .string "Hello MEXE"
    .text
    .globl  main
    .type   main, @function
main:
.LFB0:
    .cfi_startproc
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    leaq    .LC0(%rip), %rdi
    movl    $0, %eax
    call    printf@PLT
    movl    $0, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
    .size   main, .-main
    .ident  "GCC: (Debian 10.2.1-6) 10.2.1 20210110"
    .section    .note.GNU-stack,"",@progbits
```

assembly

Object File

Linker

Hello.o

Object File

```
ELF>?@@
UH??H?=???]?Hello MEXEGCC: (Debian 10.2.1-6)
 10.2.1 20210110zRx
W
                    A?C
??      $hello.cmain_GLOBAL_OFFSET_TABLE_pri
ntf????????

          ???????? .symtab.strtab.shstrtab.
rela.text.data.bss.rodata.comment.note.GNU-s
tack.rela.eh_frame @@
&\\1\
```

a.out

```
ELF>P@`9@8
          @@@@h???hh??   XX?-?=?=HP?-?=?=???
?DDP?td    <<Q?tdR?td?-?=?=/lib64/ld-linux-x8
6-64.so.2GNUF^???????? ???GNU??e?m? [ j*prin
tf__cxa_finalize__libc_start_mainlibc.so.6GL
IBC_2.2.5_ITM_deregisterTMCloneTable__gmon_s
tart___ITM_registerTMCloneTable)ui      3?@?
?((@???????????@H?H??/H??t??H????5?/?%?/@?%?/h
?H?=???f/?DH?=?/H??/H9?tH?>/H??t ?????H?=y/H?
5r/H)?H??H???H??H?H??tH?/H????fD???=9/u/UH?=
?.H??t
      H?=/?-????h????/]?????{???UH??H?=?????
???]??f.?DAWL?=,AVI??AUI??ATA??UH?-p,SL)?H??s
???H??t?L??L??D??A??H??H9?u?H?[]A\A]A^A_??H?
H??Hello MEXE<????@????@???X%?????P??????????zRx
 ????+zRx
        $???? FJ
W               ??;+3$*Dx??\U???A?C
D|`???]B?I?E ?E(?D@?H8?G@jBA0A(B BB?x???@?)
???????@
?
 @P?    ??????op???o???o\???o?=6(@GCC: (Debi
an 10.2.1-6) 10.2.1 20210110??@?p       ?
P

@P?  P ?=?=?=??@ @@??
                        ??!??@C?=j@v?=??????T
!????=??=??=? ?@?
                    ? g  @-@@
                              ?4Ge @r ?(@? ?`]?8
```

ELF

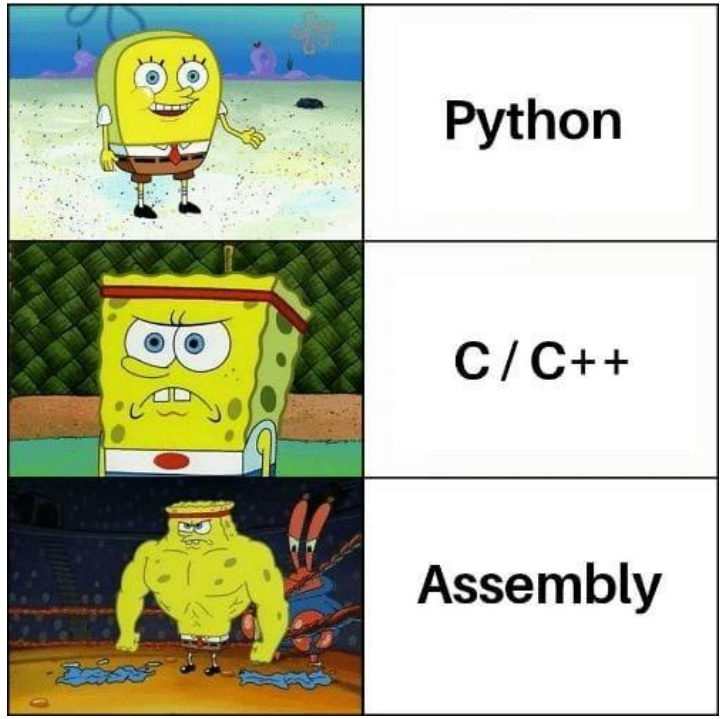
Compiler GCC (Linux)



- = GNU Compiler Collection
- Kód v jazyce C do spustitelných souborů
- volby příkazového řádku gcc (vypne se většina ochranných mechanismů !!)
  - -o: outfile
  - -m32: 32bitový spustitelný soubor (musí být nainstalován gcc-multilib)
  - -fno-stack-protector: vypne ochranu zásobníku
  - -z execstack: vytvoří spustitelný zásobník
- Příkaz terminálu:
  - gcc -m32 -fno-stack-protector -z execstack -o program program.c

Assembly jazyk

- Nízkoúrovňový (strojový) programovací jazyk
  - Různé jazyky assembleru
    - Specifické pro architekturu počítače
    - Např.: Intel, AT&T
- Rychlý přímý přístup k HW
- Používání mnemotechnik pro instrukce
  - INC EAX: zvýší hodnotu registru EAX o 1



STEP 01 Write the code

STEP 02 Assemble the code

STEP 03 Link the code

STEP 04 Load into the memory

STEP 05 Execute the program

Assembly jazyk – Intel vs. AT&T syntaxe

- Hlavní rozdíly
  - Pořadí parametrů:
    - Intel: Cíl před zdrojem — `mov eax, 5`
    - AT&T: Zdroj před cílem — `mov $5, %eax`
  - Velikost parametrů:
    - Intel: Odvozeno od názvu registru — `add esp, 4`
    - AT&T: Doplněno písmenem označující velikost operandů — `addl $4, %esp`
  - Sigily:
    - Intel: Assembler automaticky detekuje typ symbolů — `mov eax, 5`
    - AT&T: Hodnoty s předponou „$“, registry s předponou „%“ — `mov $5, %eax`

Assembly jazyk – Intel vs. AT&T syntaxe

Intel	AT&T
lea ecx,[esp+0x4]	lea 0x4(%esp),%ecx
and esp,0xfffffff0	and $0xfffffff0,%esp
push DWORD PTR [ecx-0x4]	pushl -0x4(%ecx)
push ebp	push %ebp
mov ebp,esp	mov %esp,%ebp

Assembly - základní instrukce

- MOV
- PUSH
- POP
- CALL
- RET
- JMP
- CMP
- XOR
- NOP
- INC
- ADD
- SUB

Assembly instrukce - MOV

- Syntaxe: mov DESTINATION, SOURCE

- Přesunutí aktuální hodnoty do registru
  - **mov EAX, 0x7**
    - EAX je nyní hexadecimální 7
- Přesunutí hodnoty z jednoho registru do druhého
  - **mov EAX, EBX**
    - EAX má nyní stejnou hodnotu jako EBX

Assembly instrukce - PUSH

- Syntaxe: push REGISTR

- pushneme obsah registru na zásobník
  - **push EAX**
    - Výsledek: hodnota v EAX je nyní na prvním místě zásobníku

Assembly instrukce - POP

- Syntaxe: pop REGISTR

- popneme položku z prvního zásobníku do registru
  - **pop EBX**
    - Výsledek: položka na prvním místě zásobníku se nám přesune do EBX

Assembly instrukce - CALL

- Syntaxe: call FUNKCE nebo call ADRESA

- Volá funkci
  - Instrukce CALL provádí dvě operace:
    - Posune return adresu (adresu bezprostředně za příkazem CALL) na zásobník
    - Změní EIP na call destination. Tím fakticky přenese řízení na cíl volání a zahájí jeho provádění

Assembly instrukce - RET

- Syntaxe: ret
- Vrátí hodnotu zpět callerovi

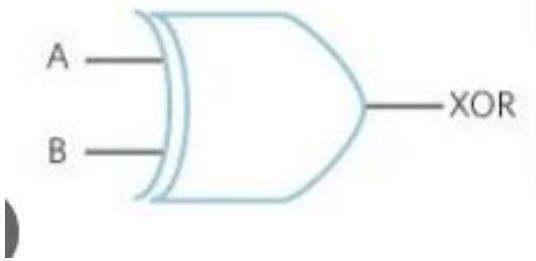
Assembly instrukce - JMP

- Syntaxe: jmp REGISTR nebo jmp ADRESA
- Skočí na zadanou adresu paměti
  - **jmp ESP**
    - skočí na adresu uloženou v registru ESP
- Podmíněné skoky
  - provedou se pouze tehdy, je-li splněna podmínka (=příznak stavu)
    - JZ: skok, pokud je nula
    - JNZ: skok, pokud není nula
    - JE: skok, pokud se rovná
  - **cmp EAX,EBX; je 0x11111111**
    - skok na adresu 0x11111111 pouze v případě, že EAX a EBX mají stejné hodnoty

Assembly instrukce - XOR

- Syntaxe: xor DESTINATION, HODNOTA
- Logická operace XOR
- Nejčastější použití: vymazání hodnoty registru
  - **xor EAX, EAX**
    - EAX se nastaví na 0

$$X = A \oplus B$$



A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Assembly instrukce - NOP

- Syntaxe: nop
- Nedělej nic (No OPeration)
- 0x90

Assembly instrukce - INC, ADD, SUB

INC

- Syntaxe: inc REGISTR
- Zvýšení hodnoty v registru o 1
  - **inc EAX**
    - EAX je nyní EAX+1

ADD, SUB

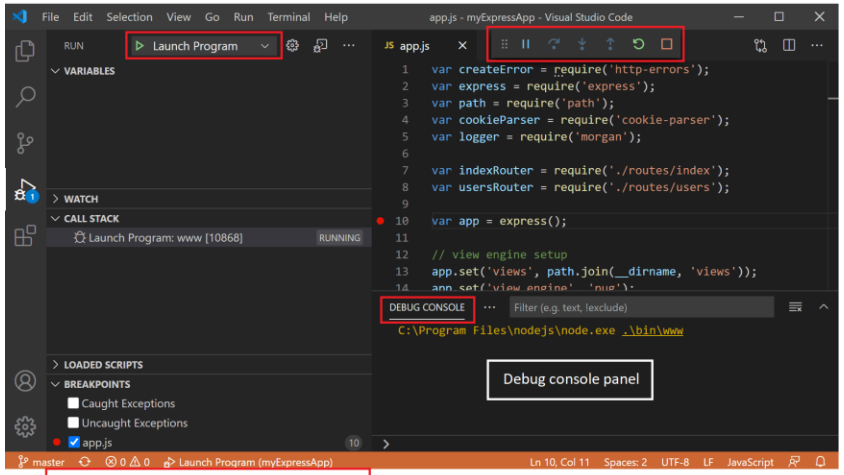
- Syntaxe: add/sub DESTINATION, HODNOTA
- + / -
  - **add EAX, 0x7**
    - EAX je nyní EAX+0x7
  - **add EAX, EBX**
    - EAX je nyní EAX+EBX

Debugger

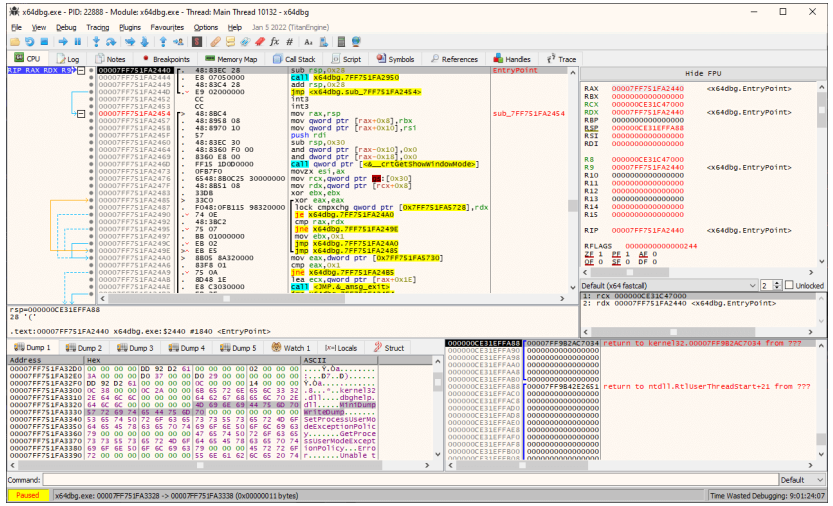
- Program používaný k testování a ladění jiných programů
- Co umí:
  - Spuštění nebo zastavení cílového programu v bodech přerušení
  - Zobrazení obsahu paměti nebo registrů procesoru
  - Modifikování obsahu paměti nebo registrů
- Důležité pro nás (pro explotaci):
  - Kontrola prováděných instrukcí
  - Zjištění stavu paměti a registrů v určitém čase
  - Zjistění, jak je zpracován vstup

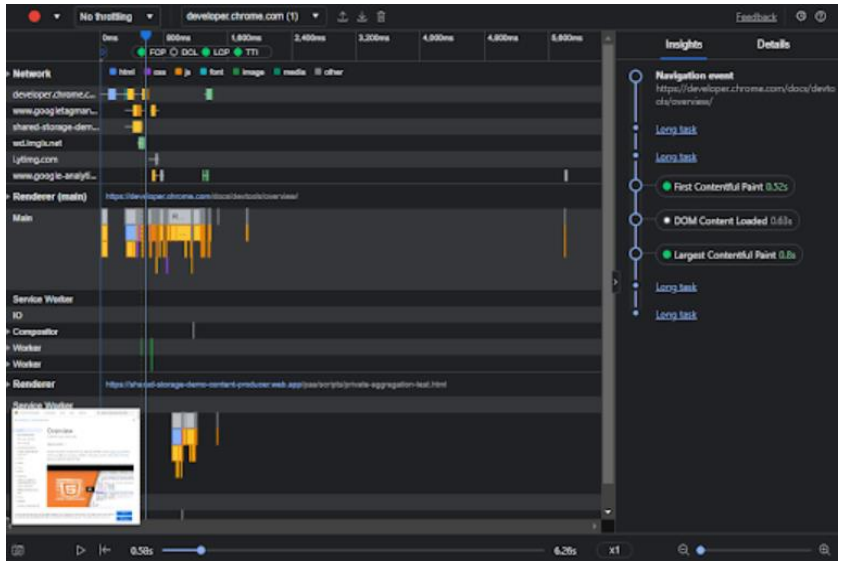
Start debugging

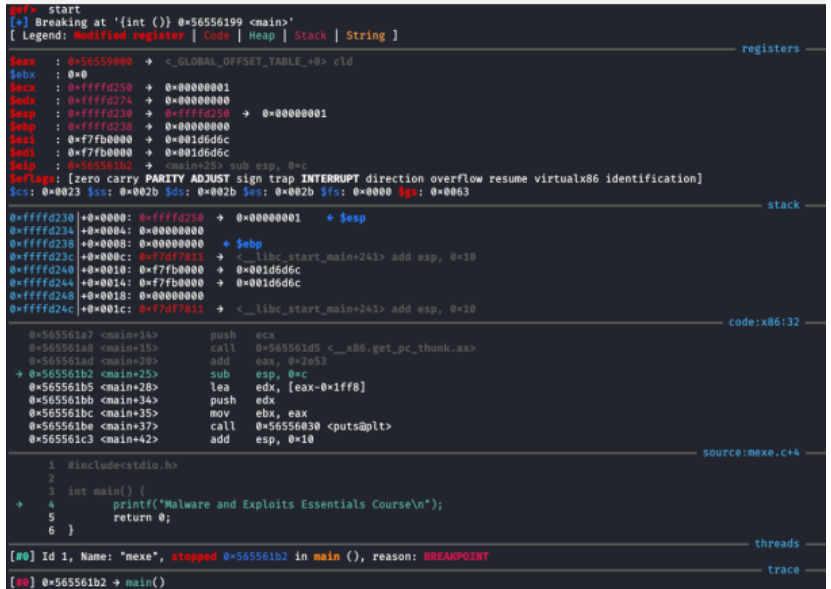
Pause, step over, step in/out, restart, stop



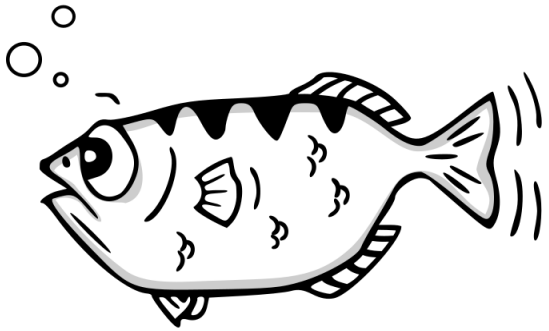
Debug side bar







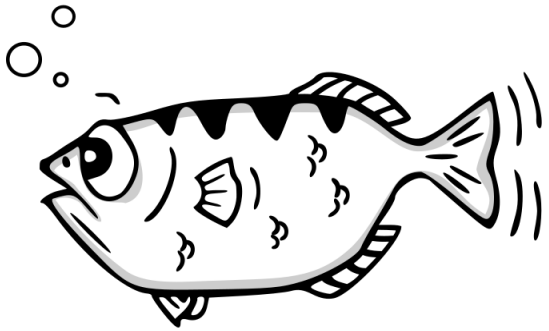
GDB - GNU Debugger



- Výchozí ladicí program pro většinu systémů Linux
- Rozhraní CLI (příkazový řádek)
- Lokální nebo vzdálené ladění
- Obsahuje i rozšíření pro specifické účely

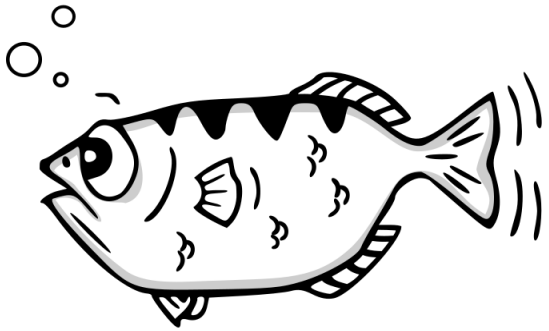
- Budeme používat:
  - GEF (GDB Enhanced Features)

GDB - Důležité příkazy



- gdb ./program (z terminálu)
  - Spuštění programu v gdb
- help
- run
  - spustí program
- start
  - spustí program a přeruší ho ve funkci main()
- si
  - Single Instruction
- ni
  - Next Instruction (krokování přes func calls)

GDB - Důležité příkazy 2

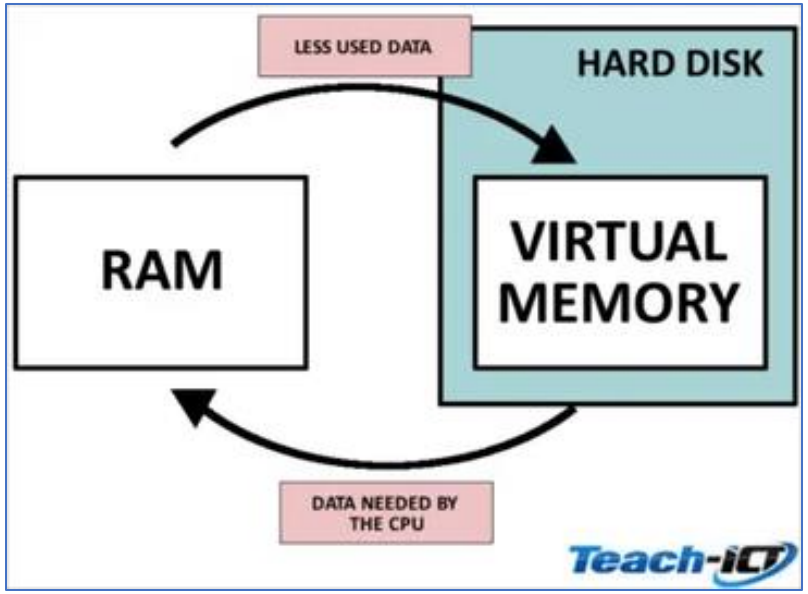


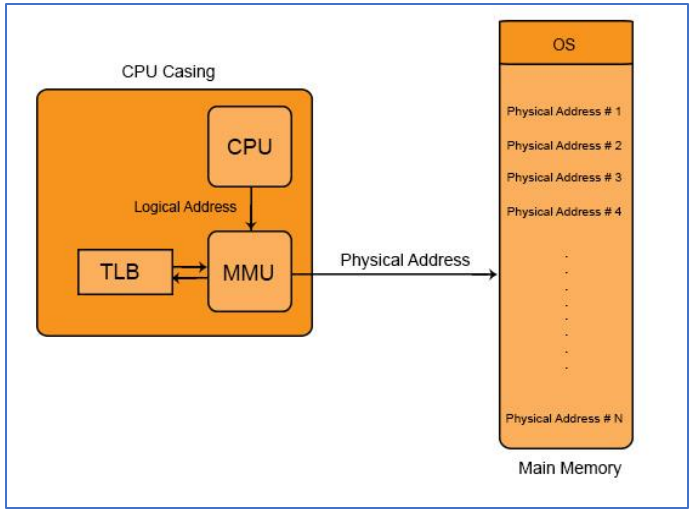
- break
  - Nastavení bodu přerušení na určitém místě
- delete
  - Smaže bod(y) přerušení
- disassemble
  - Rozebere zadaný úsek paměti
- x
  - Prozkoumá paměť (x/c)
- continue
  - pokračování v debuggování po breakpointu

Virtuální paměť

- Idealizovaná abstrakce paměti RAM pro procesy
  - dodatečná paměť navíc

- Každý proces má svou vlastní virtuální paměť
- OS mapuje virtuální adresy na HW adresy
  - MMU provádí překlad za chodu PC
    - Memory Management Unit





Ring 0-3

- Mechanismus ochrany dat a funkcí

- Ring 0: Prostor jádra (nejvíce oprávnění)
- Ring 3: Uživatelský prostor (nejmenší oprávnění)

- Systémová volání pro vyžádání služby od kernelu
  - Linux: man 2 <funkce>
  - Windows: abstrakce pomocí API

